

Template Program - Code Explanation (Hinglish)

Yeh file explain karti hai ki **"Templates"** aur **"Operator Overloading"** ka use karke C++ mein ek hi function (`add`) se different data types (int, string, custom classes jaise File, Paragraph) ko kaise combine kiya ja sakta hai. Isse Code Reusability (DRY principle - Don't Repeat Yourself) milti hai.

Sir ko impress karne ke liye main point: Sir ko batana ki isme humne "Templates" aur "Operator Overloading" ka use kiya hai taaki ek hi `add` function se hum numbers, strings, files, aur images sabko combine kar sakein.

1. `combiner.h`

Yeh header file mein ek template function aur kuch custom classes (File, Paragraph, Image) banayi gayi hain. Har class ke andar '+' operator ko apne hisab se kaam karne ke liye badla gaya hai (Operator Overloading).

```
#ifndef COMBINER_H
#define COMBINER_H

#include <string>

using namespace std;

// Yeh ek generic template function hai jo do same type ki values ko add
// karta hai.
// Template ka matlab: T ek variable type hai, jo run-time par int, string,
// ya File ban sakta hai.
template <typename T>
T add(const T &a, const T &b) {
    return a + b; // Jo bhi type hoga, uska '+' operator call hoga.
}

// File class jo file contents (text) ko store karti hai aur do file
// contents ko combine karne ki permission deti hai.
class File{
public:
    string content; // File ka data isme save hoga

    // Constructor: Jab nayi File banegi, toh uska content set karega
    File(string c) : content(c) {}

    // '+' operator overloading: Normally '+' numbers ko jodta hai,
    // par yahan humne bataya hai ki agar do File objects ke beech '+' aaye
    // toh kya karna hai.
    // Logic: Pehli file ka content, phir ek nayi line ("\n"), aur uske
    // baad dusri file ka content.
    File operator+(const File &other) const
    {
        return File(this->content + "\n" + other.content);
    }
}
```

```
};

// Paragraph class jisme text store hota hai. (Bilkul File jaisa logic)
class Paragraph{
public:
    string text;

    Paragraph(string t) : text(t) {}

    // '+' operator overloading: Do paragraphs ko nayi line se combine
    karta hai.
    Paragraph operator+(const Paragraph &other) const
    {
        return Paragraph(this->text + "\n" + other.text);
    }
};

// Image class jo image ka data (ya uska path string format me) store karti
hai.
class Image{
public:
    string imageData;

    Image(string data) : imageData(data) {}

    // '+' operator overloading: Do images ki details ko jodne ke liye,
    // unke beech mein " [COMBINED WITH] " likh deta hai.
    Image operator+(const Image &other) const
    {
        return Image(this->imageData + " [COMBINED WITH] " +
other.imageData);
    }
};

#endif
```

2. main.cpp

Yeh main file `combiner.h` ko use karti hai. Yahan alag-alag data types ko same `add()` function mein daal kar test kiya gaya hai.

```
#include <iostream>
#include <string>
#include <fstream>    // File read/write karne ke liye
#include <sstream>    // File ke content ko string mein convert karne ke
liye

using namespace std;

// Apni banayi hui file ko include karna jisme saara logic (Template) rakha
hai
```

```
#include "combiner.h"

int main()
{
    // --- 1. INTEGER ADDITION ---
    int num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    // Yahan add() function integer type accept karega aur normal addition
    dega.
    cout << "Numbers sum: " << add(num1, num2) << "\n\n";

    cin.ignore(); // Enter key ka buffer saaf karne ke liye taaki agli line
    me dikkat na ho

    // --- 2. STRING CONCATENATION (Strings ko jodna) ---
    string str1, str2;
    cout << "Enter first string: ";
    getline(cin, str1);
    cout << "Enter second string: ";
    getline(cin, str2);
    // Yahan wahi same add() function ab Strings ko ek sath jod dega
    (concatenate).
    cout << "String concatenation: " << add(str1, str2) << "\n\n";

    // --- 3. CHARACTER ADDITION ---
    char c1, c2;
    int temp_c2;
    cout << "Enter a character: ";
    cin >> c1; // Example: 'A'
    cout << "Enter an integer to add to the character: ";
    cin >> temp_c2; // Example: 2
    c2 = temp_c2;
    // Char addition: 'A' (ASCII 65) + 2 = 67. (char) laga kar wapas 'C'
    print hoga.
    cout << "Char addition ('" << c1 << " + " << temp_c2 << "): " <<
    (char)add(c1, c2) << "\n\n";

    cin.ignore();

    // --- 4. FILE COMBINATION (Custom Class) ---
    // Files ko padh kar unko combine karna
    string fileName1, fileName2;
    cout << "Enter path/name for File 1: ";
    getline(cin, fileName1);
    cout << "Enter path/name for File 2: ";
    getline(cin, fileName2);

    // File 1 read karne ka code
    ifstream f1(fileName1);
    string fileContent1;
    if (f1) {
```

```
        stringstream buffer;
        buffer << f1.rdbuf(); // Poori file ek baar me buffer me daal li
        fileContent1 = buffer.str(); // Buffer se string nikal li
    } else {
        cout << "Could not open File 1. Using empty content.\n";
    }

    // File 2 read karne ka code
    ifstream f2(fileName2);
    string fileContent2;
    if (f2) {
        stringstream buffer;
        buffer << f2.rdbuf();
        fileContent2 = buffer.str();
    } else {
        cout << "Could not open File 2. Using empty content.\n";
    }

    // Yahan hamari Custom File class ban rahi hai.
    File file1(fileContent1);
    File file2(fileContent2);
    // Jab add(file1, file2) call hoga, tab File class ka overloaded '+'
operator chalega.
    File combinedFile = add(file1, file2);
    cout << "Combined Files Content:\n"
        << combinedFile.content << "\n\n";

    // --- 5. PARAGRAPH COMBINATION ---
    string para1, para2;
    cout << "Enter text for Paragraph 1: ";
    getline(cin, para1);
    cout << "Enter text for Paragraph 2: ";
    getline(cin, para2);
    Paragraph p1(para1);
    Paragraph p2(para2);
    // Paragraph class ka overloaded '+' chalega
    Paragraph combinedParagraph = add(p1, p2);
    cout << "Combined Paragraphs:\n"
        << combinedParagraph.text << "\n\n";

    // --- 6. IMAGE COMBINATION ---
    string imgPath1, imgPath2;
    cout << "Enter source path for Image 1: ";
    getline(cin, imgPath1);
    cout << "Enter source path for Image 2: ";
    getline(cin, imgPath2);
    Image image1(imgPath1);
    Image image2(imgPath2);
    // Image class ka overloaded '+' chalega aur " [COMBINED WITH] " beech
me aayega.
    Image combinedImage = add(image1, image2);
    cout << "Combined Image Sources:\n"
        << combinedImage.imageData << "\n";
```

```
    return 0;  
}
```

3. file1.txt & file2.txt

Yeh dono dummy text files hain jo upar diye program mein file combine option ko test karne ke liye banai gayi hain.